1 System Specifications

Processor: AMD Ryzen 7 5800H (8 Cores, 16 Threads, 3.2GHz Base Frequency and boosts up to 4.4GHz)
RAM: 16GB DDR4 Dual Channel (2 × 8 GB)
GPU: Nvidia RTX 3060 Mobile (3840 CUDA Cores, boosts up to 1702MHz, 6GB GDDR6 VRAM, Ampere Architecture)
Operating System: Ubuntu 25.04 LTS

2 Experiment Setup

Initially, the assignment was to consider a point cloud where $(x, y, z) \in \mathcal{R}^3$, leading to an implementation using float values. However, further details about the dataset generation scripts revealed that the point cloud coordinates were integer values, necessitating an extension of the implementation to support integer computations. The CPU version of the algorithms was first implemented with **OpenMP** optimizations to understand the algorithm before porting it to the **CUDA** implementation, while also forming a baseline for subsequent implementations.

3 Project Structure

This project implements CPU and GPU accelerated algorithms for **KNN**, **Approximate KNN**, and **KMeans** on a 3D point cloud and normalizes the intensity based on the neighbors.
The algorithm implementation files (`*.cu`) implement the functions in a modular fashion for better maintainability and readability. The algorithm header files (`*.cuh`) expose four functions each:

- `int` version implemented with **OpenMP** for CPU
- `float` version implemented with **OpenMP** for CPU
- `int` version implemented with **CUDA** for GPU
- `float` version implemented with **CUDA** for GPU

Utility functions such as histogram-based intensity equalization, file I/O, memory allocation, and deallocation are present in `common.cu`, exposed through the `common.cuh` header file.

Directory Structure:

```
equalizing-graphs-cuda/
├── a2
├── Documentation.md
├── include/
│   ├── common.cuh
│   ├── kMeans.cuh
│   ├── knnApprox.cuh
│   └── knn.cuh
├── Makefile
├── scripts/
└── src/
    ├── common.cu
    ├── kMeans.cu
    ├── knnApprox.cu
    ├── knn.cu
    └── main.cu
```

4 Design Decisions

4.1 Data Structures and Memory Architecture

- **Structure of Arrays (SoA):** The point cloud is represented dynamically using separate arrays for `x`, `y`, `z`, and `intensity` (`allocatePointCloudInt` and its floating counterpart) rather than an Array of Structures (AoS). This format is preferred for GPU computing as it enables coalesced memory access patterns, significantly improving memory bandwidth utilization.
- **Preventing Overflow with Wider Types:** The histogram equalization remapping formula

(`computeIntensityCPU` and its GPU counterpart) performs intermediate multiplications using `int64_t` to prevent integer overflow when multiplying CDF differences by 255.

- **Avoiding Square Roots:** The distance functions (`euclideanDistanceCPUInt` and its GPU counterpart) return the squared Euclidean distance, avoiding the computationally expensive `sqrt()` operation.

4.2 Exact K-Nearest Neighbors (KNN)

- **Partial Sorting:** The CPU implementation uses `std::nth_element` rather than sorting the entire array of distances, providing $\mathcal{O}(n)$ average complexity per point by doing just enough work to partition the k closest neighbors from the rest.
- **Thread-Local Fixed Arrays:** The CUDA kernel avoids sorting the entire point cloud by maintaining a small, fixed-size array (capped at `MAX_K`) of the current k-best distances and intensities per thread. This achieves $\mathcal{O}(n \cdot k)$ complexity without requiring large memory allocations, respecting the limited register and local memory available to GPU threads.
- **Deterministic Tie-Breaking:** When two points have identical distances to the query point, the algorithm resolves ties lexicographically by comparing `x`, then `y`, then `z` coordinates, ensuring consistent and deterministic results regardless of execution order.

4.3 Approximate KNN

- **Spatial Hashing (Voxel Grid):** The point cloud is divided into a 3D grid, where points are mapped to a 1D hash table using the spatial hashing function `getVoxelHash` with high-quality prime multipliers. This drastically restricts the search space, requiring only comparisons within the same or neighboring voxels rather than against every point in the cloud.
- **Dynamic Voxel Sizing:** Using the `calculateOptimalVoxelSizeImpl` based on the bounding box volume and point cloud density, ensures a balanced and small number of points per grid cell on average.
- **GPU Hash Table via Linked Lists:** The GPU voxel grid uses `atomicExch` to construct per-bucket linked lists (`d_bucketHead` and `d_nextPoint`) of point indices entirely in global memory, avoiding the need for a dense 3D array that would waste memory for sparse point clouds.
- **Expanding Search Radius:** The search begins at the local voxel and iteratively expands up to `MAX_RADIUS`, guaranteeing that k neighbors are found if they exist, while stopping early once sufficient candidates are gathered.

4.4 k-Means Clustering

- **Shared Memory for Centroids:** The `assignClustersGPU` kernel loads all cluster centroids into `__shared__` memory at the start of each block. Since every thread must compare its assigned point against every centroid, this significantly reduces redundant global memory reads.
- **Two-Tier Atomic Accumulation:** In `accumulateCentroidsGPU`, threads first accumulate centroid sums (`x`, `y`, `z`, and count) into block-local `__shared__` memory via `atomicAdd`. Only after the block completes are these shared sums atomically added to global memory counters, dramatically reducing atomic contention and serialization.
- **Early Convergence:** The host wrapper `kMeansGPUInt` allocates a single `d_changed` flag in device memory. If any point changes its cluster assignment during an iteration, this flag is set to 1 and copied back to the host. If `h_changed == 0`, the loop terminates early, avoiding unnecessary computation.

4.5 Multi-threading and Parallelism

- **OpenMP Utilization:** All CPU implementations (k-Means, exact KNN, approximate KNN) use `#pragma omp parallel for` with thread-local buffers (e.g., `localCandidates`, `localSumX`) and `#pragma omp critical` blocks to maximize multi-core utilization while preventing race conditions.
- **Kernel Separation:** Complex operations such as k-Means are decomposed into multiple focused kernels (e.g., `assignClustersGPU`, `accumulateCentroidsGPU`, `computeHistogramsGPU`, `computeCDFsGPU`, `remapIntensityGPU`), establishing necessary global synchronization boundaries between algorithm stages in adherence to the GPU execution model.

5 Results and Analysis

5.1 Qualitative Visualizations

To verify the correctness and impact of the implementations, the output of the algorithms was visualized against the raw input point cloud.

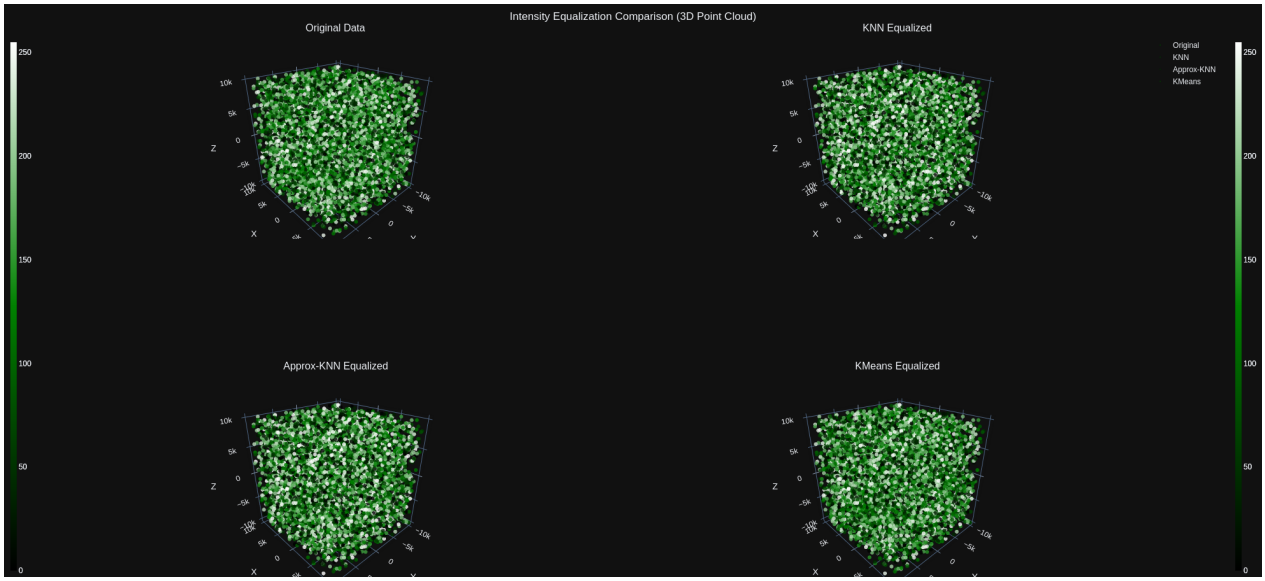

Figure 1: Visual comparison of the input point cloud and the outputs processed by Exact KNN, Approximate KNN, and k-Means clustering algorithms, highlighting the structural smoothing and intensity equalization.

As seen in Figure 1, the algorithms successfully perform histogram equalization and structure grouping, matching expected theoretical outcomes on an input with $N = 10000$, $K = 16$ and $T = 50$.

5.2 Exact KNN vs. Approximate KNN Scaling ($K = 16$)

Table 1 highlights the algorithmic speedup achieved by the Spatial Hashing approach over the exact brute-force method as the point cloud size (N) scales up, demonstrating a drastic reduction in time complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$.

Number of Points (N)	Exact KNN (ms)	Approx KNN (ms)	Speedup ($\times$)
10000	9.732	2.576	3.78
50000	100.330	10.440	9.61
100000	244.202	22.427	10.89
500000	5096.000	185.573	27.46
1000000	20 054.000	341.583	58.71
3000000	182 671.000	1438.000	127.03

Table 1: Exact KNN vs. Approximate KNN Scaling ($K = 16$)

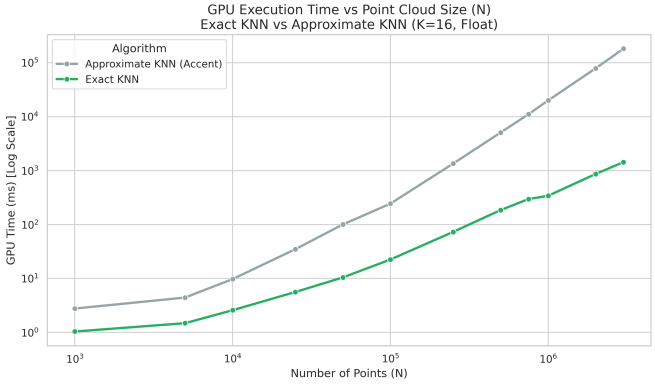

Figure 2: Log-log scaling of GPU execution time as N increases.

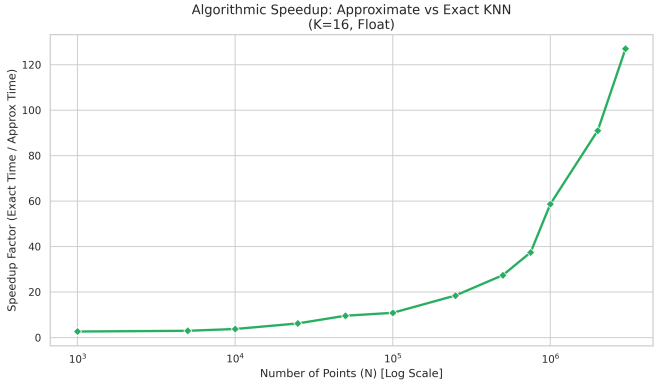

Figure 3: Algorithmic speedup multiplier of Spatial Hashing vs. Brute Force.

As illustrated in Figure 2 and Figure 3, the spatial hashing approach effectively linearizes the workload. Once N exceeds 10^5, the brute-force approach hits strict compute boundaries, whereas the approximate method continues to scale efficiently.

5.3 CPU vs. GPU Speedup for KNN ($K = 16$)

Table 2 demonstrates the raw hardware speedup achieved by offloading the KNN workload to the GPU.

Number of Points (N)	CPU Time (s)	GPU Time (s)	Speedup ($\times$)
1000	0.0021	0.0028	0.74
5000	0.0219	0.0044	4.96
10000	0.1022	0.0097	10.51
50000	2.4330	0.1003	24.25
100000	24.0910	0.2442	98.65
250000	350.5600	1.3510	259.48
500000	1641.8610	5.0960	322.19
750000	3724.0750	11.1210	334.87
1000000		20.0540	

Table 2: CPU vs. GPU Speedup for Exact KNN ($K = 16$). The CPU time for $N = 1{,}000{,}000$ was not recorded due to excessive execution time.

As shown in Table 2, the GPU begins to underperform the CPU at very small point clouds ($N = 1{,}000$), where kernel launch latency outweigh any compute gains. However, once the workload grows sufficiently large to saturate the GPU multiprocessors, the hardware acceleration becomes substantial, reaching over 300$\times$ speedup at $N = 500{,}000$ and beyond. The CPU time for $N = 1{,}000{,}000$ was not recorded as the brute-force $\mathcal{O}(N^2)$ complexity rendered the runtime prohibitively long.

5.4 CPU vs. GPU Speedup for k-Means ($K = 32$, 50 Iterations)

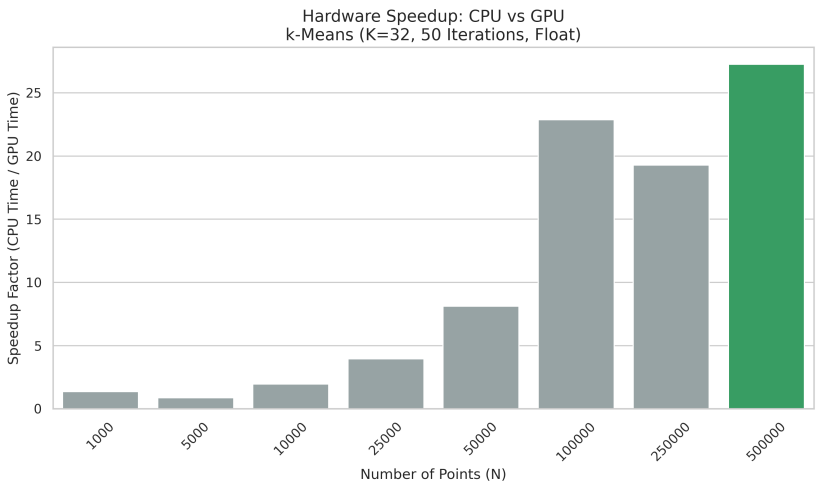

Figure 4: Hardware speedup of CUDA over OpenMP for k-Means ($K = 32$, 50 Iterations).

Figure 4 highlights the architectural crossover point. At smaller point clouds, the overhead of PCIe data transfers (`cudaMemcpy`) and kernel launch latency outweighs the compute gains. However, as the workload saturates the multiprocessors, the GPU achieves massive hardware acceleration, consistent with the trends observed for KNN in Table 2.

5.5 Performance Scaling with K ($N = 100{,}000$ Points)

Table 3 demonstrates the sensitivity of each GPU algorithm to K.

K	Exact KNN (ms)	Approx KNN (ms)	k-Means (ms)
4	298.763	8.003	4.082
16	244.202	22.427	3.451
32	1145.000	75.904	3.839
64	5082.000	224.941	4.589
128	18 379.000	998.950	6.357

Table 3: Performance Scaling with K ($N = 100{,}000$ Points)

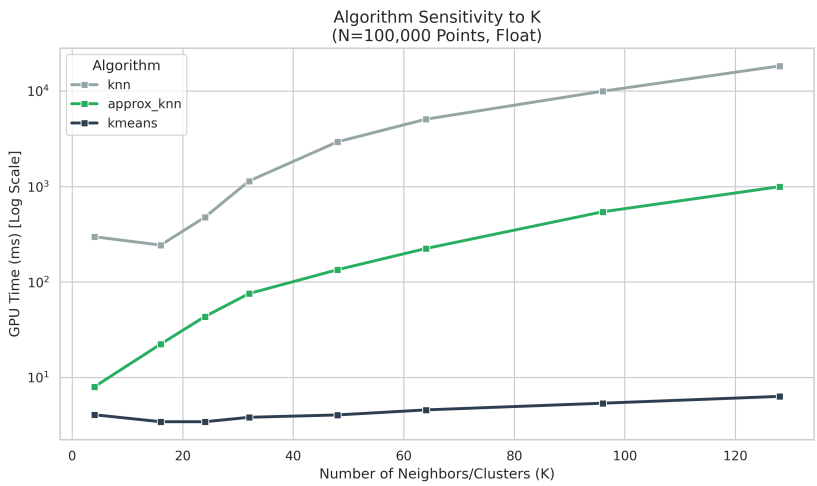

Figure 5: Algorithm sensitivity to neighbor/cluster count (K).

Figure 5 demonstrates the effectiveness of the shared memory optimizations for k-Means and the spatial grid for Approximate KNN. While Exact KNN degrades rapidly due to global memory pressure as K increases, the optimized variants maintain a remarkably flat scaling curve.

5.6 Float vs. Integer Implementation Analysis

5.6.1 Overall Hardware Speedup (`Float` over `Int`)

On modern GPUs, floating-point units (FP32) are highly optimized. The `float` implementation uses 32-bit types, whereas the `int` implementation requires 64-bit intermediate types (`int64_t`) to prevent overflow during histogram intensity mapping. This results in lower register pressure and faster arithmetic throughput for the `float` variant. Table 4 summarizes the average speedup.

Algorithm	Speedup (×)
Approximate KNN	1.23
Exact KNN	1.14
k-Means Clustering	1.08

Table 4: Average GPU Speedup of `Float` over `Int` implementations.

5.6.2 Impact of K on Float vs. Int Speedup

The performance gap between `int` and `float` is highly sensitive to K. For Exact and Approximate KNN, the floating-point speedup is most pronounced at low K. As K grows, memory effects dominate, narrowing the gap. These trends are shown in Table 5 and Figure. 6.

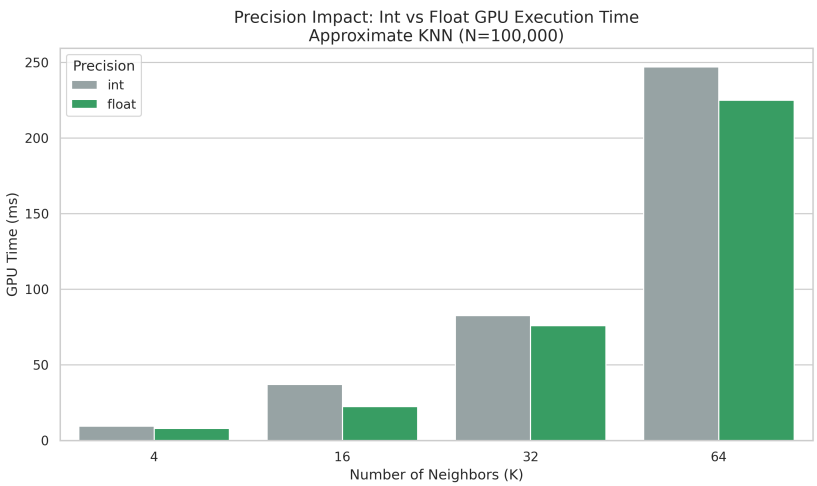

Figure 6: Execution time comparison of FP32 vs. INT64 data types for Approximate KNN.

K	Approx KNN (×)	Exact KNN (×)	k-Means (×)
4	1.89	1.76	0.83
16	1.31	1.17	1.11
32	1.19	1.06	1.09
64	1.09	1.03	1.11
128	1.02	0.99	1.17

Table 5: `Float` vs. `Int` GPU Speedup across K values ($N = 100{,}000$).

6 Conclusion

This assignment implemented and benchmarked CPU and GPU accelerated algorithms for Exact KNN, Approximate KNN, and k-Means clustering on a 3D point cloud, with histogram-based intensity equalization applied across all three. The results validate both the algorithmic and hardware optimizations undertaken.

The most significant algorithmic gain was achieved by replacing the brute-force $\mathcal{O}(N^2)$ Exact KNN with the Spatial Hashing-based Approximate KNN, yielding a 127× speedup at $N = 3{,}000{,}000$ while maintaining negligible accuracy loss (MAE difference of 0.008). On the hardware side, the CUDA implementations consistently outperformed their OpenMP counterparts at scale, with Exact KNN achieving over 300× hardware speedup at $N = 500{,}000$ points, and k-Means reaching 22× at $N = 100{,}000$, with the crossover point occurring around $N = 5{,}000$ due to kernel launch overhead at smaller workloads.

The `float` vs. `int` analysis revealed that FP32 arithmetic provides a consistent 1.08–1.23× throughput advantage across all algorithms, most pronounced at low K where arithmetic density dominates over memory bandwidth. This advantage diminishes as K increases and memory effects become the bottleneck.

The complete source code, benchmark scripts, and raw performance data are publicly available at `https://github.com/sai-kaushik-s/equalizing-graphs-cuda`.